Betsy -- Decision Log Summary

The story of building an autonomous procurement agent, one decision at a time.

Thread: Data -> Agents -> Design -> Integration -> Approval Flow -> Learning

---

DL-01 -- We needed somewhere to test

`DL-01-mock-data-gap-analysis.md`

Started with nothing. Built a realistic mock environment -- 12 SKUs, 6 suppliers, 15 invoices, 4 injectable scenarios, a FastAPI server, and a live dashboard. The dashboard showed everything a procurement manager would need. But it just showed it. SKU-003 critically low, the right supplier right there, nothing connecting them.

Decision: Build the mock environment as the test bed before touching any agent code. GAP -> DL-02: Data exists. Nothing acts on it.

DL-02 -- How do you architect an autonomous agent?

`DL-02-agent-architecture-comparison.md`

Two approaches built and compared: Pipeline (sequential 6-stage handoff) vs Orchestra (3 parallel agents with a coordinator). Tested both against the same 4 scenarios. Found the pipeline fails when two things go wrong simultaneously -- the orchestra handles it with a hardcoded precedence table. Kept both.

Decision: Both architectures stay -- Pipeline for simple single-issue runs, Orchestra for competing simultaneous conditions. GAP -> DL-03: Agents work in the terminal. Nobody else can see them running or acting.

DL-03 -- How do you show an AI to someone who's never used one?

`DL-03-ui-design-user-requirements.txt`

Jenny can't read a Python stack trace. Researched enterprise AI tool patterns (Salesforce Einstein, Copilot, Tableau Pulse) -- they all put AI as a layer on top of familiar data, not a separate app. Designed around that: command bar, plain English narrative, AI-scored data tables, inline approval cards that explain WHY Betsy stopped before asking approve or decline. That explanation is the trust mechanism.

Decision: Enterprise AI layer pattern -- two separate HTML files served from the same server (betsy.html for Jenny, index.html for dev). GAP -> DL-04: Design is wired to live data but approval buttons go nowhere.

DL-04 -- First real run, and what "autonomous" actually means

`DL-04-implementation-testing.txt`

Wired betsy.html to the live API, added POST /api/run-agent as a background task, switched to llama3.1:8b for better JSON output. Ran the stockout scenario for real. Agent detected two conditions, both decisions came back pending human review. First reaction: the bot did nothing. Then looked at the code -- the PO was $11,937, hard limit is $5,000. The safeguard worked. An agent that escalates a $12k order is exactly right.

Decision: llama3.1:8b as the model, /api/run-agent as the trigger, DRY_RUN and MAX_AUTO_USD as env vars. GAP -> DL-05: Approval queue is documented but the endpoint doesn't exist. Jenny can't act on escalated decisions.

DL-05 -- Closing the loop: HITL approval queue end-to-end

`DL-05-hitl-approval-flow.txt`

Built /api/approvals -- GET pending, POST approve/reject. act.py queues every requires_human decision to it with the full PO payload pre-built. Approve executes the deferred PO directly in state. 11/11 test cases passed. Bug found: state.reset() was wiping the approval queue after every pipeline run -- fixed by keeping approvals independent of scenario state.

Decision: Store the full payload at queue time, execute on approval. No second LLM call, no state drift. GAP -> DL-06: Approvals live in memory. Server restart = queue gone. Jenny can't trust pending decisions to survive.

DL-06 -- Making memory permanent

`DL-06-persistence-learning.txt`

Three gaps closed in one sprint: SQLite persistence (agent_log + approvals survive server restarts), EMA supplier scoring (on-time delivery raises reliability score, late delivery lowers it, formula ?=0.2), and APScheduler (pipeline fires automatically every 30 minutes, next_run visible in betsy.html stats panel). Restart test confirmed zero data loss. Formula confirmed to four decimal places against known values.

Decision: SQLite via built-in sqlite3, BackgroundScheduler in a scheduler_instance.py singleton (avoids circular import with stats.py), EMA triggered on PATCH .../delivered with alpha=0.2. GAP -> DL-07: Scores update correctly in isolation. But does a better score actually change what Betsy orders?

DL-08 -- Does Betsy tell Jenny when it matters?

`DL-08-notifications.txt`

Zero notification infrastructure existed before this DL. Jenny had to keep betsy.html open and wait for a 5-second poll. Now Betsy pushes desktop toast notifications (via plyer) and HTML emails (via stdlib smtplib) the moment any of four events occur: approval needed, auto-approved PO placed, supplier score crossing the warning threshold, or duplicate invoice flagged. All notification code lives server-side -- no pipeline changes. Notification failure is fully contained and never interrupts the agent. 22 unit tests, all passing.

Decision: Desktop + email via plyer + smtplib. Settings panel added to betsy.html. Runtime config via POST /api/notifications/config, backed by mutable module-level attributes in server/config.py. GAP -> future: Runtime config resets on server restart -- users should set .env vars for persistence.

DL-07 -- Does Betsy actually get smarter?

`DL-07-long-term-learning.txt`

The EMA formula is correct -- DL-06 proved that. This DL proves the consequence: that delivery history changes real procurement decisions, not just numbers. Built test_long_term_learning.py -- two real LLM pipeline runs, 8 delivery rounds between them. QuickShip gets 5 × 8d-late (0.92 -> 0.44), FastParts gets 3 × on-time (0.95 -> 0.97). Composite crossover at round 5. Score trajectory confirmed exactly. Full before/after pipeline comparison in progress.

Decision: Build an integration test with two real LLM calls and a structured delivery schedule that produces a mathematically predictable composite crossover. Status: Score evidence solid ? -- behavioral comparison (pipeline run 1 vs run 2 supplier choice) in progress ?

Updated as each DL is completed. Screenshots go in `decision_logs/images/`.